

GalaxyGate + RunPod track

Scan for the guide, or open the repository

<https://github.com/GalaxyGate/philly-hackathon>

Coffee & Code AI Agent Hackathon



Before you start

- A laptop, a GitHub account, and an email inbox you can open now.
- If you are a team, one teammate does Part 1 and shares the resulting URL and repository with the others.
- **Cursor.** Desktop: download from <https://cursor.com/download>, sign in, then File, Open Folder, and choose a new empty folder named `hackathon`, created on your Desktop first. Web: open <https://cursor.com/agents>; cloud agents need a paid Cursor plan. If you received a Cursor credit code card at the GalaxyGate table, redeem it at the link on the card first.
- **Claude Code.** Desktop app or terminal, opened in an empty folder named `hackathon`. Web: <https://claude.ai/code>, connected to a GitHub repository of yours; any empty repository works.
- **Codex.** Terminal, opened in an empty folder named `hackathon`; step 4 is written for the terminal. Codex on the web cannot connect MCP servers.

If you have none of these, use Cursor.

Step 2. GalaxyGate account

1. Open <https://dash.galaxygate.net/-/register>. If the coupon box is empty, type the coupon code from the GalaxyGate table.
2. Open the verification email and click the link. Check spam if it does not arrive within a minute.
3. Sign in at <https://dash.galaxygate.net>. You should see one workspace and no servers.

If you already had a GalaxyGate account before the event, sign in and ask at the GalaxyGate table to apply the coupon.

Coupon:

❶ Optional. Applies credit to your new account.

Step 3. RunPod account and one API key

1. Sign up at <https://console.runpod.io/signup>.
2. Open the RunPod credit link from your check-in email and redeem it. Your balance should read \$15.
3. Settings, API Keys, Create API Key. Name it `hackathon-app`, permission `ALL`, and copy it now. RunPod shows it once. It goes into the prompt in step 5.

The key lives in your app's environment on your server, never in your code or repository. It also lands in the chat history and in tool-call logs, and anyone in your GalaxyGate workspace can see it, so keep the workspace to your team.

The agent needs `ALL` once, to create your endpoint. After that you can create a second key with only the permission your app needs and ask the agent to put that one in the app's environment; the Secure score rewards this.

Step 4. Cursor: MCP servers

Both servers sign you in through a browser window. There is no key to paste. The two addresses are `https://mcp.galaxygate.net/mcp` and `https://mcp.getrunpod.io/`.

Desktop. In Cursor's Explorer sidebar click New File, type `.cursor/mcp.json` as the whole file name, press Enter, and paste this. File Explorer and Finder refuse names starting with a dot. If the file already exists, add the two entries inside its `mcpServers` object.

```
{
  "mcpServers": {
    "galaxygate": { "url": "https://mcp.galaxygate.net/mcp" },
    "runpod": { "url": "https://mcp.getrunpod.io/" }
  }
}
```

Step 4. Cursor: sign in

- Save, restart Cursor, open Customize in the sidebar, then MCP.
- Each server shows Needs login. Click it, approve in the browser tab, and check its toggle is on.
- Web. At <https://cursor.com/agents> add each address as an HTTP server in the MCP dropdown, and sign in.

Step 4. Claude Code: MCP servers

Desktop or terminal. In a terminal opened in `hackathon`:

```
claude mcp add -t http galaxygate https://mcp.galaxygate.net/mcp  
claude mcp add -t http runpod https://mcp.getrunpod.io/
```

Then start `claude` in that same folder, type `/mcp`, and complete the sign-in for each server. The servers are saved for this folder, so always start Claude Code from it.

Web. At <https://claude.ai> open Customize, Connectors, Add custom connector; add each address and sign in. Then start a session on your repository at <https://claude.ai/code>, turn both connectors on, and set network access to Full.

Step 4. Codex: MCP servers

In a terminal opened in `hackathon`:

```
codex mcp add galaxygate --url https://mcp.galaxygate.net/mcp
codex mcp login galaxygate
codex mcp add runpod --url https://mcp.getrunpod.io/
codex mcp login runpod
```

Codex blocks network access from commands unless you allow it. Add these lines to `~/.codex/config.toml`, creating the file if it is missing, then start `codex`:

```
sandbox_mode = "workspace-write"

[sandbox_workspace_write]
network_access = true
```

Step 4. Check the connection

```
Call the GalaxyGate list_workspaces tool and the RunPod  
list-endpoints tool and show me both results.
```

- One GalaxyGate workspace with your name on it, and a RunPod endpoint list, which may be empty.
- If either call is denied, redo the sign-in for that server.

Step 5. Create your server: paste this

Replace `yourteam` with your team name: 3 to 32 characters, lowercase letters, digits, and hyphens only, starting and ending with a letter or digit. Replace the key. Then paste the whole block into your tool.

```
TEAM_NAME=yourteam  
RUNPOD_API_KEY=rpa_paste-your-key-here
```

Fetch

```
https://raw.githubusercontent.com/GalaxyGate/philly-hackathon/main/AGENT.md,  
with curl if you have no tool for web pages, and follow it step by step.  
Stop and tell me if it does not load.
```

If your tool cannot open web pages, open <https://github.com/GalaxyGate/philly-hackathon/blob/main/AGENT.md> yourself, copy its whole text, and paste it under the two lines instead.

Step 5. What the agent does

- It creates your server, installs Docker and builds the demo on it, creates a vision-model endpoint on RunPod under your account, deploys the demo with a public name, runs one real generation, and writes `HACKATHON.md` in your folder with every id you will need.
- The build and the first generation each take a few minutes. The first generation downloads the model onto the GPU and takes several minutes; later ones are much faster.
- Stay with the laptop while this runs. Cursor, Claude Code and Codex all stop and wait for you to approve the commands that reach your own server, and none of them says anything while it waits. If you come back to a screen that has not moved, look for an approval prompt before you assume something broke.
- Done when the agent reports the health check and one real generation passed, and your URL opens on your phone.

The one rule, and spending

- Your app, its data, and your RunPod key live on your GalaxyGate server. Your app's backend calls RunPod over HTTPS.
- Never call RunPod from browser JavaScript, and never put the key in your repository.
- The proxy in front of your URL drops any response with no bytes for 100 seconds, so a slow GPU call must stream or send progress bytes. The demo does this.
- The demo caps itself at 20 generations per hour, in `MAX_GENERATIONS_PER_HOUR`, and the cap counts every visitor together. Open `/api/usage` on your URL to see what is left, and ask your agent to raise the cap for the judging window if you used it up testing. Keep a cap in anything you build.
- Your endpoint scales to zero, and bills for the whole time a worker runs, including the model load and the idle minute after a request.
- The first request after a quiet period is slow while the model loads, so send one generation through your app yourself right before a judge tries it. Check your RunPod balance in the console before you submit.

The demo: Sketch to Site

- Open your URL on your phone. Photograph a website sketch drawn on paper or a whiteboard, add a line of notes if you like, and press Build the page.
- A vision model on your RunPod endpoint turns the photo into a web page. You get a link and a QR code to the live page.
- Every page is listed at `/sites`, kept on the `/data` mount, so a redeploy keeps them.

Sketch to Site[Pages](#)

Sketch to Site

Photograph a hand-drawn sketch of a web page. The model reads the drawing, writes the page, and this server hosts it.

20 of 20 generations left this hour.

Take a photo

Notes for the model (optional)

Name the business, or ask for a color

Build the page

Fork and clone

The demo on your server is `demos/sketch` in <https://github.com/GalaxyGate/philly-hackathon>. Fork it into your GitHub account. In Cursor desktop, Claude Code desktop, or Codex:

```
Clone https://github.com/<your GitHub username>/philly-hackathon into this
folder and work in demos/sketch.
```

On Claude Code web or a Cursor cloud agent, start a session on your fork, turn both MCP connections on, and paste in `HACKATHON.md`. Edit whatever you like, or add a new app folder next to the demos.

Redeploy your changes

Push to your fork, then tell your agent:

Fetch

```
https://raw.githubusercontent.com/GalaxyGate/philly-hackathon/main/AGENT.md  
and follow its step 12 for  
https://github.com/<your GitHub username>/philly-hackathon and folder  
demos/sketch.
```

If the agent cannot find `HACKATHON.md`, paste its contents into the chat. Ask
`Show me the app logs` when something is wrong, and `Restart the app` to restart it.

Use a GPU model: your endpoint

Your endpoint `hackathon-v1` runs `Qwen/Qwen2.5-VL-7B-Instruct`. Call `https://api.runpod.ai/v2/<ENDPOINT_ID>/openai/v1/chat/completions` like the OpenAI chat API, with the image as an `image_url` part holding a `data:image/jpeg;base64, ...` URL.

Give your app's call at least fifteen minutes, as the demo does, and stream or send progress bytes while it waits.

Use a GPU model: ready-made

No deployment. Tell your agent:

```
Add a route to my app that generates an image with RunPod's
Flux Schnell public endpoint using the RUNPOD_API_KEY from
the environment, saves the image under /data, and shows it on
a page.
```

It posts `{"input":{"prompt":"...", "width":768, "height":768}}` to `https://api.runpod.ai/v2/black-forest-labs-flux-1-schnell/runsync` with the key as a bearer token, width and height between 256 and 1536 and divisible by 64, and returns `output.image_url`. More: <https://docs.runpod.io/public-endpoints/overview>

Required to be judged in this track

- Your app runs on a GalaxyGate server created during the build period, and a judge can open it.
- Its GPU work runs on RunPod, and a judge's own input produces a request you can show: the RunPod console entry, or the `id` from the response.
- Starter code and RunPod's public models are allowed; say what you reused.
- Fabricated evidence, including a faked RunPod call, removes the project from this track.

Both integrations are the entry ticket. Having them earns nothing; how well your app uses them is what the points below measure.

Submit on Devpost

Submit at <https://coffee-and-code-agent.devpost.com/> before the deadline shown there, select the GalaxyGate prize, and alongside the standard fields give judges:

- Your app URL, one exact input to try, and the instance id and app id from `HACKATHON.md`.
- Your RunPod endpoint id or public model, plus one request record as above.
- Your repository, the commit you started from, the commit you submitted, and what you changed.
- A video under three minutes and a README that lets a stranger run it. Check both open signed out.

Do not include environment screens, API keys, private keys or credit links in anything you submit.

Scoring, 100 points plus up to 15 extra

Judges score live, using their own inputs, not your demo input.

- **It works, 30.** Their first input produces the promised result (15). A second, different input also works (10). Nothing in the judge's session ends in a crash, a blank screen, or a spinner that never resolves (5); an upstream failure the app reports clearly does not count against this.
- **Simple to use, 20.** A first-timer completes the core task with no instructions (10). The screen always says what is happening: loading, GPU waking up, done, failed (5). The README gets a stranger running in one read (5).
- **Reliable, 15.** Results survive a page reload and a second run (5). A slow start or upstream error is shown, not hidden (5). Paid routes have a cap or a login (5).

Scoring, continued, and ties

- **Secure, 15.** Secrets only in the server's environment, not in the repository, logs, or screenshots (5). Only your server calls RunPod (5). The app's key is no broader than it needs (5); one `All` key for both setup and app scores 2 of 5.
- **Scalable, 10.** GPU work on an endpoint that scales to zero and can add workers (5). State outside the container, so a redeploy keeps it (5).
- **Agentic, 10.** The app decides what to do next from model output or tools, not one fixed call (10).
- **Extra credit, up to 15.** GalaxyGate, 5 each up to 10: a second server doing real work, private networking between servers, a floating IP, a scheduled backup or snapshot, or a load balancer. RunPod, 5: a second endpoint or model, a network volume, or a pod used for training or a batch job. Each must be used for a reason the judge can see.
- **Ties.** Simple to use, then Reliable. If your app is down when judges reach you, you have the rest of the judging window to bring it back; an app that never comes back is not judged in this track. If it comes back, a timestamped end-to-end recording covers only the Secure, Scalable and Agentic points, because every other category is scored from a judge's own input.

When something fails

- **Needs login, or a tool call denied.** Redo that server's sign-in: Cursor desktop in Customize, MCP; Cursor web in the MCP dropdown; Claude Code desktop with `/mcp`; Claude Code web in Customize, Connectors; Codex with `codex mcp login <name>`.
- **The agent stopped in the middle, or your session ended.** Paste the block from step 5 again, with the same team name and the same key, in a new chat. The instructions find the server, the endpoint and the app you already have and carry on. Do not change your team name, and do not ask the agent to delete anything first.
- **The server never became ready.** Ask the agent to run `get_instance` and show the state.

When something fails, continued

- **The first deploy sticks or fails.** The instructions recover once. If it fails twice, ask for the workflow state and its failed children.
- **An error mentioning 401.** The key was not substituted or is wrong. Create a new key and ask the agent to update the app's environment, keeping every existing variable.
- **RunPod answers 402.** Your credit is gone. Check the console.
- **A generation fails or times out after a few minutes.** The first request after a quiet period loads the model onto the GPU. Wait two minutes, then send exactly one more. If that also fails, ask the agent to read the app logs and bring the error text to the GalaxyGate table.

Bring your tool, your step, the ids from `HACKATHON.md`, and the error text with your key removed. Help is at the GalaxyGate table.

After the event

- Delete the server from the GalaxyGate panel or ask your agent to.
- Delete the `hackathon-v1` endpoint in the RunPod console.
- Revoke the `hackathon-app` key there.
- Check that no other endpoint you created still has workers running.